

APPEX LEAGUE

PLAY

INTRODUCTION

Apex League adalah game kompetitif berbasis fisika dimana pemain mengendalikan kendaraan bertenaga roket dalam arena dua dimensi yang sepenuhnya datar (flat). Fokus utama permainan adalah penguasaan momentum dan arah vektor untuk menggiring bola ke gawang lawan.

Game ini dirancang sebagai bukti kompetensi rekayasa perangkat lunak, dengan mengedepankan stabilitas mesin fisika. Tidak ada elemen ketinggian (sumbu-Z) dalam game ini, sehingga seluruh tantangan berpusat pada akurasi benturan dan timing manuver pada bidang datar.

CORE GAMEPLAY MECHANICS

Dash

Boost

Drift

Demolition



TECH STACK & ARCHITECTURE

Frontend (Client): Java menggunakan framework LibGDX + Box2D Physics Engine.

Backend (API Server): Spring Boot (Java) sebagai RESTful API handler.

Primary Database (Storage): PostgreSQL untuk integritas data persisten (User Profile, Car Stats, Match History).

In-Memory Cache (Speed): Redis (Sorted Sets) untuk komputasi Global Leaderboard secara instan.

DUAL-DATABASE STRATEGY FOR LEADERBOARD

Masalah: Mengurutkan ribuan data pemain di SQL konvensional sangat membebani server saat diakses terus-menerus.

Solusi Apex League: PostgreSQL difokuskan untuk mengamankan data riwayat secara ACID compliance.

- Saat pertandingan selesai, Backend secara otomatis mengirim nilai ke Redis ZSET (Sorted Set).
- Hasilnya: Data untuk tab MMR, WINS, GOALS, SAVES, dan DEMOS dapat di-fetch dalam hitungan milidetik.

DESIGN PATTERN IMPLEMENTATION

Factory Method Pattern

Implementasinya ada di CarFactory.java dan ArenaFactory.java. Keduanya memusatkan proses pembuatan objek Box2D yang cukup kompleks ke satu tempat, jadi entity lain tidak perlu tahu detail fixture, body, dan shape yang dipakai. Ini membuat pembuatan mobil dan arena lebih konsisten dan lebih mudah diubah.

Strategy Pattern

Implementasi paling jelas ada di MovementSystem.java. Sistem ini menukar perilaku handling antara grip normal dan drifting lewat nilai seperti FULL_GRIP dan DRIFT_GRIP, lalu mengubah respons tenaga, putaran, dan traksi tanpa mengubah struktur entity. Hasilnya, cara mobil bergerak bisa berganti secara dinamis sesuai input pemain.

State Pattern

Pola ini terlihat lewat pengelolaan state permainan di GameManager.java dan alur screen di PlayScreen.java. Kode memakai flag seperti isKickoff, isResetting, isGameOver, dan isPaused untuk mengubah perilaku game sesuai fase pertandingan. Jadi walaupun tidak dibuat sebagai kelas-kelas state terpisah, perilakunya tetap mengikuti konsep state yang berganti berdasarkan kondisi runtime.

Observer Pattern

Pola ini tampak di GameContactListener.java, yang bereaksi ketika terjadi kontak fisika seperti goal, demolition, boost pad, dan save. Listener ini lalu memperbarui GameManager, HUD, dan PlayScreen tanpa objek-objek itu saling bergantung langsung pada source event fisika. Secara praktik, ini adalah observer-style callback yang memisahkan sumber event dari responsnya.

DESIGN PATTERN IMPLEMENTATION

Factory Method Pattern

Implementasinya ada di CarFactory.java dan ArenaFactory.java. Keduanya memusatkan proses pembuatan objek Box2D yang cukup kompleks ke satu tempat, jadi entity lain tidak perlu tahu detail fixture, body, dan shape yang dipakai. Ini membuat pembuatan mobil dan arena lebih konsisten dan lebih mudah diubah.

Strategy Pattern

Implementasi paling jelas ada di MovementSystem.java. Sistem ini menukar perilaku handling antara grip normal dan drifting lewat nilai seperti FULL_GRIP dan DRIFT_GRIP, lalu mengubah respons tenaga, putaran, dan traksi tanpa mengubah struktur entity. Hasilnya, cara mobil bergerak bisa berganti secara dinamis sesuai input pemain.

State Pattern

Pola ini terlihat lewat pengelolaan state permainan di GameManager.java dan alur screen di PlayScreen.java. Kode memakai flag seperti isKickoff, isResetting, isGameOver, dan isPaused untuk mengubah perilaku game sesuai fase pertandingan. Jadi walaupun tidak dibuat sebagai kelas-kelas state terpisah, perilakunya tetap mengikuti konsep state yang berganti berdasarkan kondisi runtime.

Observer Pattern

Pola ini tampak di GameContactListener.java, yang bereaksi ketika terjadi kontak fisika seperti goal, demolition, boost pad, dan save. Listener ini lalu memperbarui GameManager, HUD, dan PlayScreen tanpa objek-objek itu saling bergantung langsung pada source event fisika. Secara praktik, ini adalah observer-style callback yang memisahkan sumber event dari responsnya.

DESIGN PATTERN IMPLEMENTATION

Command Pattern

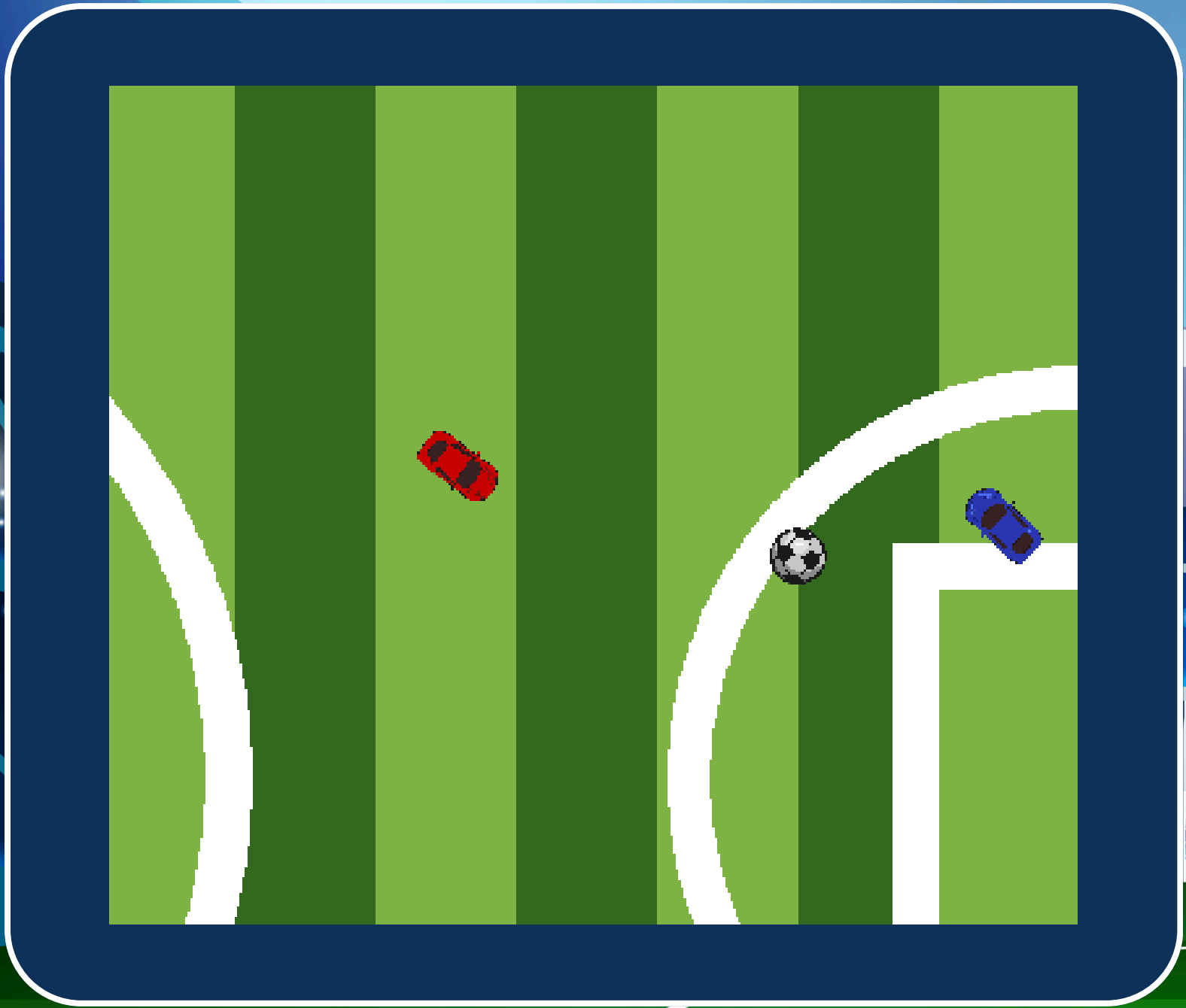
Implementasinya ada di `Command.java` dan kelas turunan seperti `MoveCommand.java`, `JumpCommand.java`, `BoostCommand.java`, dan `DriftCommand.java`. Input keyboard di `InputSystem.java` diubah menjadi objek command lalu dieksekusi ke `InputComponent`. Ini membuat input lebih rapi dan gampang diperluas ke replay atau jaringan.

Component Pattern

Pola ini adalah fondasi arsitektur gameplay lewat class seperti `InputComponent.java`, `PhysicsComponent.java`, `JumpComponent.java`, dan `TextureComponent.java`. Entity seperti `PlayerEntity.java` dibentuk dari komponen-komponen kecil yang fokus pada data tertentu. Jadi perilaku mobil tidak ditaruh dalam satu objek besar, tetapi dipecah ke data + sistem yang memprosesnya.

Object Pool Pattern

Implementasinya ada di `BoostPadPool.java` dan dipakai oleh `BoostManager.java`. Objek boost pad dibuat, dipakai, lalu dikembalikan ke pool supaya tidak terus-menerus membuat objek baru. Cara ini membantu menekan alokasi memori dan mengurangi beban garbage collector saat game berjalan.



GLOBAL LEADERBOARD

MMR	WINS	GOALS	SAVES	DEMOS
RANK	USERNAME	SCORE		
1	DREAMER7948	875		
2	PROPLAYER0533	830		
3	SPEEDY8834	705		
4	NIGHTOWL2787	700		
5	DREAMER3402	700		
6	THUNDER2776	690		

BACK



READY FOR
KICKOFF?